

01

Lab 1.1 — Build it, then break it

Module 1 · 90 minutes

Wrong answers are the data. Don't fix them — write them down.

1

Build

a chatbot, ~40 lines

2

Break

find where it is wrong

3

Record

transcripts → Checkpoint 1.1

Today LLM alone → **Lab 1.2** add retrieval → **Capstone** your call

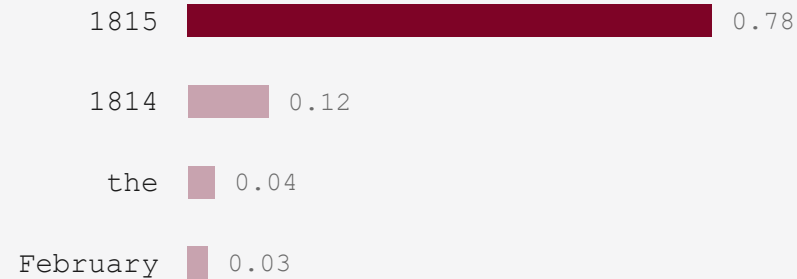
02

Where the answer comes from

One mechanism, running over and over

It predicts the next token — nothing else

"The War of 1812 ended in ____"



Sample one.
Append it.
Repeat.

Not in the loop:

✗ look anything up

✗ cite a source

✗ check the claim

Fluent and correct come out of the same machine. Tone is not evidence.

03

Four ways it fails

Step 4 — one probe each, save the transcript

1

Hallucination

Invents specifics. Same confident tone.

Try: "Summarize the 2019 Kestrel-Lin retrieval study."

2

Missing knowledge

Never saw your private data.

Try: "What is our PTO carryover policy?"

3

Stale knowledge

Training stopped on a date.

Try: "What changed in the latest LangChain release?"

4

Inconsistency

Reword it — the answer moves.

Try: Ask one fact two ways, in two separate runs.

Prompting changes tone · format · hedging

Only retrieval adds knowledge it never had

04 Memory is not knowledge

What actually goes over the wire on turn 2

USE_HISTORY = False

Turn 1 →

system

user · When was the War of 1812?

Turn 2 →

system

user · And who won?

turn 1 is gone

Bot: "Could you clarify which competition?"

USE_HISTORY = True

Turn 1 →

system

user · When was the War of 1812?

Turn 2 →

system

user · When was the War of 1812?

assistant · June 1812 - Feb 1815

user · And who won?

re-sent

Bot: "A stalemate — the Treaty of Ghent restored the pre-war borders."

The API is stateless. "Memory" is a list you re-send — and re-pay for — every turn. It fixes **context**, not **knowledge**.

05 The two functions you write

lab_1_1_chatbot_starter.py

Steps 1–2 · no memory

```
# one question in, one answer out
prompt = ChatPromptTemplate.from_messages([
    ("system", SYSTEM_PROMPT),
    ("human", "{question}"),
])
chain = prompt | llm

def respond(user_input):
    reply = chain.invoke({"question": user_input})
    return reply.content
```

Step 3 · with memory

```
# the conversation lives here, in your process
history = [SystemMessage(content=SYSTEM_PROMPT)]

def respond(user_input):
    history.append(HumanMessage(content=user_input))
    reply = llm.invoke(history)
    history.append(AIMessage(content=reply.content))
    return reply.content
```

One call sends a question. **The other sends the whole conversation.**

Step 4 · flip `USE_HISTORY`, run the two-turn test, then go hunting for the four failure modes.